Analyse Exploratoire des Données (EDA) - Instacart Market Basket

> **Objectif** : Explorer le jeu de données d'achats Instacart (comportement des utilisateurs, fréquences de commande, produits les plus vendus, taux de réachat) et extraire des **insights stratégiques et business pertinents**.
>
> **Fichiers inclus** :
>
> - `orders.csv` : Historique des commandes (3,4M de commandes, jours, heures, délais)
> - `order_products.csv` : Détail des paniers (32,4M d'articles commandés, ordre d'ajout, réachats)
> - `products.csv` : Catalogue des produits (49,688 produits)
> - `aisles.csv` : Rayons du magasin (134 rayons)
> - `departments.csv` : Départements du magasin (21 départements)

1. Configuration & Chargement Optimisé des Données

Afin de traiter efficacement 32,4 millions d'enregistrements sans engorger la mémoire RAM, nous définissons des types de données optimisés (`int8` , `int16` , `int32` , `float32`).

In [1]:
```
import warnings
warnings.filterwarnings('ignore')
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os

# Paramètres généraux des graphiques
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams['font.size'] = 11
plt.rcParams['figure.titlesize'] = 14
plt.rcParams['axes.titlesize'] = 12
plt.rcParams['figure.dpi'] = 100

DATA_DIR = 'dataset'

# Dtypes optimisés pour économiser jusqu'à 75% de RAM
orders_dtypes = {
    'order_id': 'int32',
```

```
    'user_id': 'int32',
    'order_number': 'int16',
    'order_dow': 'int8',
    'order_hour_of_day': 'int8',
    'days_since_prior_order': 'float32'
}

order_products_dtypes = {
    'order_id': 'int32',
    'product_id': 'int32',
    'add_to_cart_order': 'int16',
    'reordered': 'int8'
}

products_dtypes = {
    'product_id': 'int32',
    'aisle_id': 'int16',
    'department_id': 'int16'
}

aisles_dtypes = {'aisle_id': 'int16'}
departments_dtypes = {'department_id': 'int16'}

print("Chargement des datasets...")
orders = pd.read_csv(os.path.join(DATA_DIR, 'orders.csv'), dtype=orders_dtyp
order_products = pd.read_csv(os.path.join(DATA_DIR, 'order_products.csv'), d
products = pd.read_csv(os.path.join(DATA_DIR, 'products.csv'), dtype=product
aisles = pd.read_csv(os.path.join(DATA_DIR, 'aisles.csv'), dtype=aisles_dtyp
departments = pd.read_csv(os.path.join(DATA_DIR, 'departments.csv'), dtype=d

print("✅ Chargement terminé avec succès !")
```

```
Chargement des datasets...
✅ Chargement terminé avec succès !
```

2. Aperçu Général & Qualité des Données

In [2]:

```
summary_data = []

tables = {
    'orders': orders,
    'order_products': order_products,
    'products': products,
    'aisles': aisles,
    'departments': departments
}

for name, df in tables.items():
    mem_mb = df.memory_usage(deep=True).sum() / (1024 * 1024)
    summary_data.append({
        'Table': name,
        'Lignes': f"{len(df):,}",
        'Colonnes': df.shape[1],
        'Taille RAM (Mo)': f"{mem_mb:.2f}"
    })
```

```
summary_df = pd.DataFrame(summary_data)
display(summary_df)

print("\n--- Diagnostic des Valeurs Manquantes ---")
for name, df in tables.items():
    nulls = df.isnull().sum()
    nulls_filtered = nulls[nulls > 0]
    if not nulls_filtered.empty:
        print(f"Table '{name}' :")
        for col, count in nulls_filtered.items():
            pct = (count / len(df)) * 100
            print(f"  - {col}: {count:,} valeurs manquantes ({pct:.2f}%)")
    else:
        print(f"Table '{name}' : Aucune valeur manquante.")
```

	Table	Lignes	Colonnes	Taille RAM (Mo)
0	orders	3,421,083	6	52.20
1	order_products	32,434,489	4	340.25
2	products	49,688	4	4.17
3	aisles	134	2	0.01
4	departments	21	2	0.00

```
--- Diagnostic des Valeurs Manquantes ---
Table 'orders' :
  - days_since_prior_order: 206,209 valeurs manquantes (6.03%)
Table 'order_products' : Aucune valeur manquante.
Table 'products' : Aucune valeur manquante.
Table 'aisles' : Aucune valeur manquante.
Table 'departments' : Aucune valeur manquante.
```

3. Analyse du Comportement Temporel des Commandes

Nous analysons quand les clients passent leurs commandes :

- **Jours de la semaine** (`order_dow` : 0 = Dimanche par convention Instacart, 1 = Lundi, etc.)
- **Heures de la journée** (`order_hour_of_day` : 0h à 23h)
- **Fréquence de réachat** (`days_since_prior_order`)

In [3]:
```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

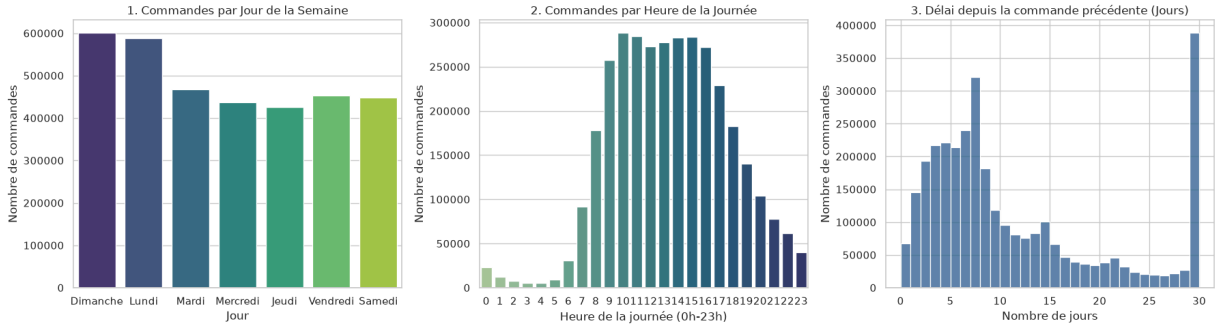
# 1. Distribution par jour de la semaine
dow_map = {0: 'Dimanche', 1: 'Lundi', 2: 'Mardi', 3: 'Mercredi', 4: 'Jeudi',
dow_counts = orders['order_dow'].value_counts().sort_index()
dow_labels = [dow_map[i] for i in dow_counts.index]
```

```
sns.barplot(x=dow_labels, y=dow_counts.values, ax=axes[0], hue=dow_labels, p
axes[0].set_title("1. Commandes par Jour de la Semaine")
axes[0].set_xlabel("Jour")
axes[0].set_ylabel("Nombre de commandes")
axes[0].ticklabel_format(style='plain', axis='y')

# 2. Distribution par heure de la journée
hour_counts = orders['order_hour_of_day'].value_counts().sort_index()
sns.barplot(x=hour_counts.index, y=hour_counts.values, ax=axes[1], hue=hour_
axes[1].set_title("2. Commandes par Heure de la Journée")
axes[1].set_xlabel("Heure de la journée (0h-23h)")
axes[1].set_ylabel("Nombre de commandes")
axes[1].ticklabel_format(style='plain', axis='y')

# 3. Délai depuis la commande précédente
sns.histplot(orders['days_since_prior_order'].dropna(), bins=30, kde=False,
axes[2].set_title("3. Délai depuis la commande précédente (Jours)")
axes[2].set_xlabel("Nombre de jours")
axes[2].set_ylabel("Nombre de commandes")

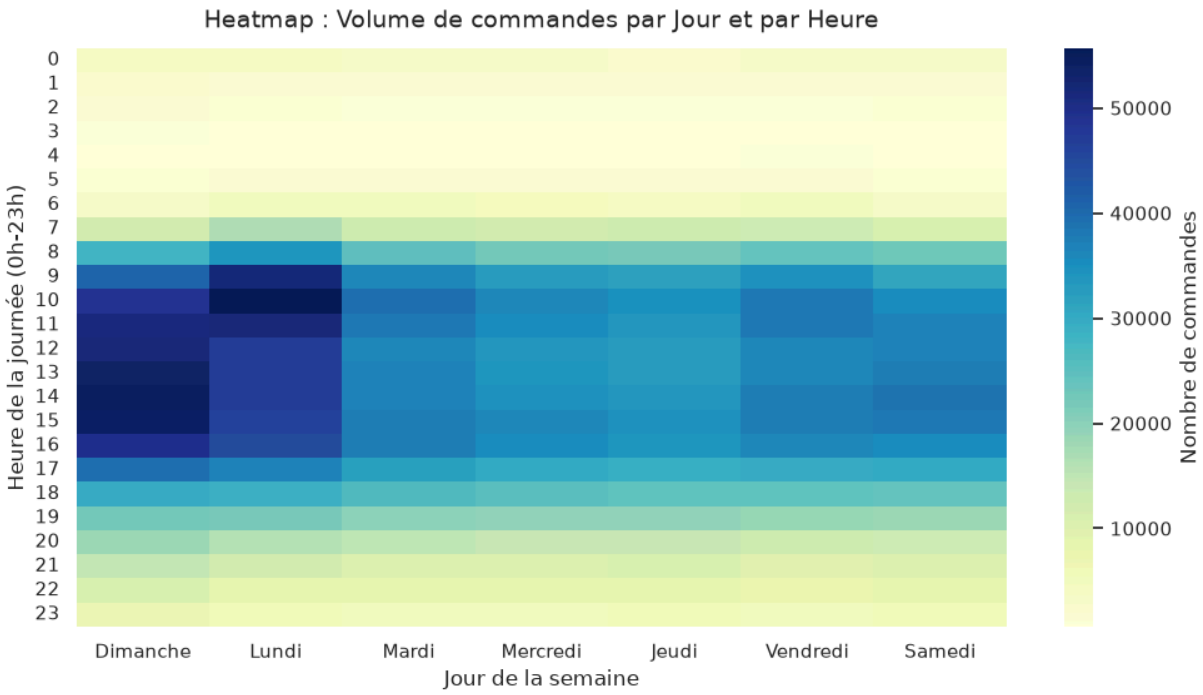
plt.tight_layout()
plt.show()
```



In [4]:

```
plt.figure(figsize=(12, 6))
dow_hour_matrix = orders.groupby(['order_dow', 'order_hour_of_day']).size().
dow_hour_matrix.columns = [dow_map[i] for i in dow_hour_matrix.columns]

sns.heatmap(dow_hour_matrix, cmap="YlGnBu", cbar_kws={'label': 'Nombre de co
plt.title("Heatmap : Volume de commandes par Jour et par Heure", fontsize=14
plt.xlabel("Jour de la semaine")
plt.ylabel("Heure de la journée (0h-23h)")
plt.show()
```



4. Analyse des Utilisateurs et de la Taille des Paniers

Analysons le comportement des 206,209 clients :

- Combien de commandes passe un utilisateur ?
- Combien d'articles contient en moyenne une commande ?

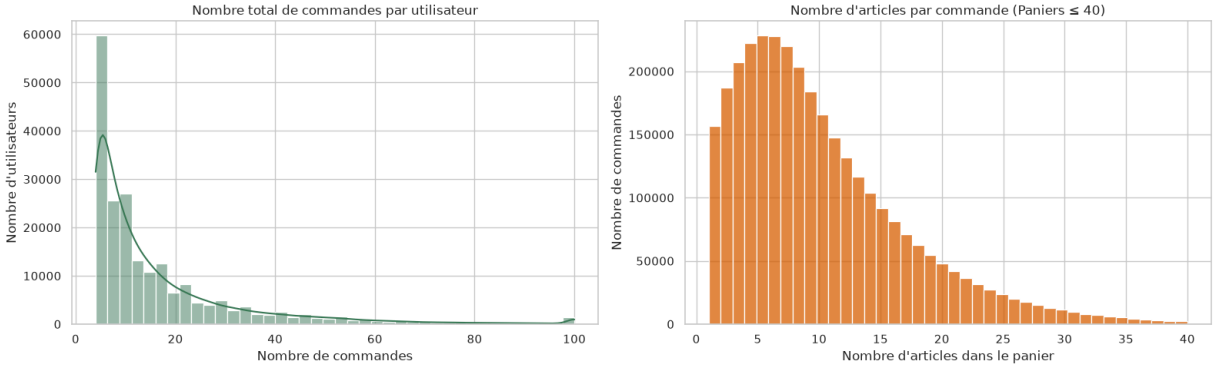
In [5]:
```
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# 1. Nombre de commandes par utilisateur
orders_per_user = orders.groupby('user_id')['order_number'].max()
sns.histplot(orders_per_user, bins=40, ax=axes[0], color="#3b7a57", kde=True
axes[0].set_title("Nombre total de commandes par utilisateur")
axes[0].set_xlabel("Nombre de commandes")
axes[0].set_ylabel("Nombre d'utilisateurs")

# 2. Nombre d'articles par panier
items_per_order = order_products.groupby('order_id')['product_id'].count()
sns.histplot(items_per_order[items_per_order <= 40], bins=40, ax=axes[1], co
axes[1].set_title("Nombre d'articles par commande (Paniers ≤ 40)")
axes[1].set_xlabel("Nombre d'articles dans le panier")
axes[1].set_ylabel("Nombre de commandes")

plt.tight_layout()
plt.show()

print("--- Statistiques de la Taille des Paniers ---")
print(f" - Taille moyenne du panier : {items_per_order.mean():.2f} articles"
print(f" - Taille médiane du panier : {items_per_order.median():.0f} article
print(f" - Panier maximum enregistré : {items_per_order.max()} articles")
```




```
--- Statistiques de la Taille des Paniers ---
 - Taille moyenne du panier : 10.09 articles
 - Taille médiane du panier : 8 articles
 - Panier maximum enregistré : 145 articles
```

5. Top Produits, Rayons (Aisles) et Départements

In [6]:

```
# Enrichissement de la table produits
products_full = products.merge(aisles, on='aisle_id').merge(departments, on=

# Top 15 Produits
top_products = order_products['product_id'].value_counts().head(15).reset_ir
top_products.columns = ['product_id', 'order_count']
top_products = top_products.merge(products_full, on='product_id')

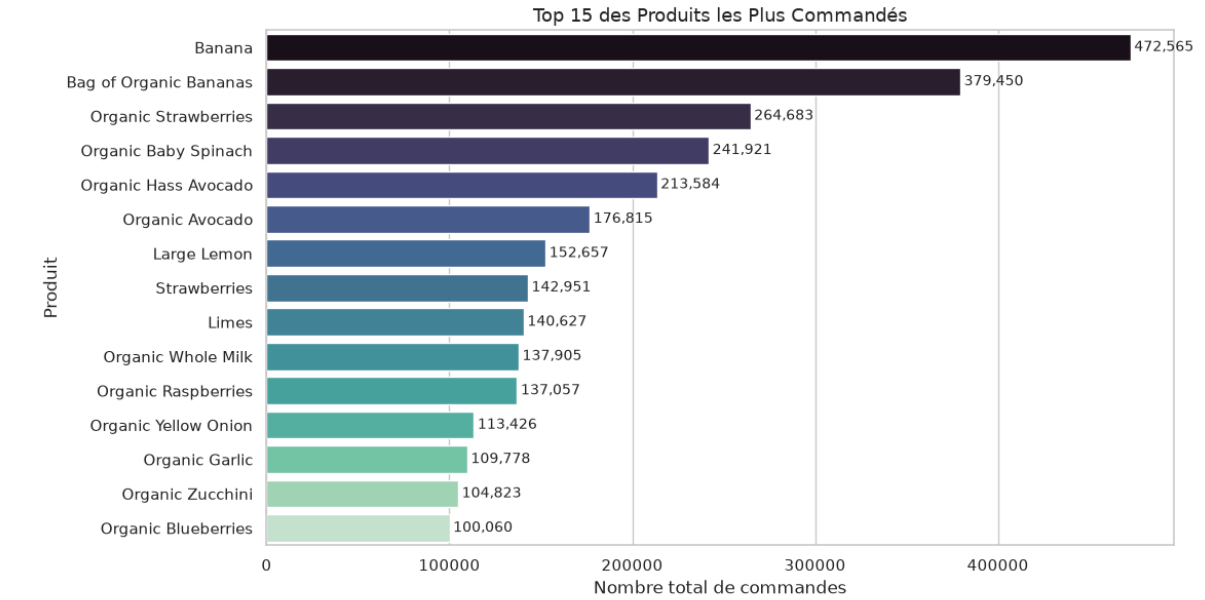
# Top 15 Rayons (Aisles)
top_aisles = order_products.merge(products[['product_id', 'aisle_id']], on='
                          .merge(aisles, on='aisle_id')['aisle'].value_cou
top_aisles.columns = ['aisle', 'order_count']

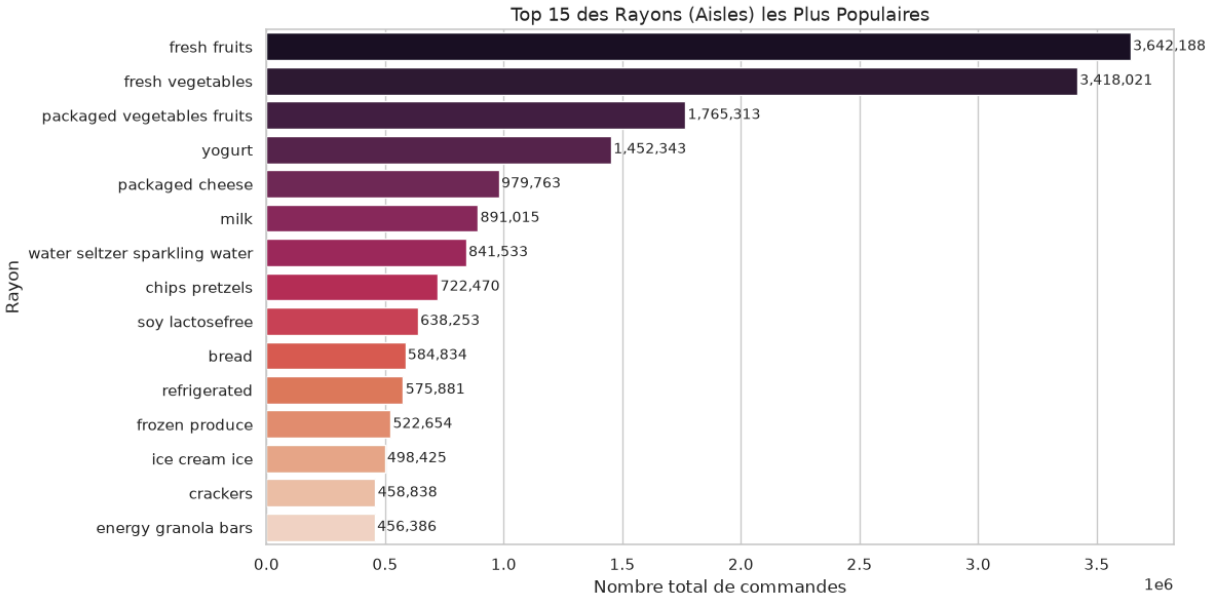
fig, axes = plt.subplots(2, 1, figsize=(12, 12))

# Plot Top Produits
sns.barplot(data=top_products, x='order_count', y='product_name', ax=axes[0]
axes[0].set_title("Top 15 des Produits les Plus Commandés", fontsize=13)
axes[0].set_xlabel("Nombre total de commandes")
axes[0].set_ylabel("Produit")
for i, v in enumerate(top_products['order_count']):
    axes[0].text(v + 2000, i, f"{v:,}", va='center', fontsize=10)

# Plot Top Rayons
sns.barplot(data=top_aisles, x='order_count', y='aisle', ax=axes[1], hue='ai
axes[1].set_title("Top 15 des Rayons (Aisles) les Plus Populaires", fontsize
axes[1].set_xlabel("Nombre total de commandes")
axes[1].set_ylabel("Rayon")
for i, v in enumerate(top_aisles['order_count']):
    axes[1].text(v + 10000, i, f"{v:,}", va='center', fontsize=10)

plt.tight_layout()
plt.show()
```

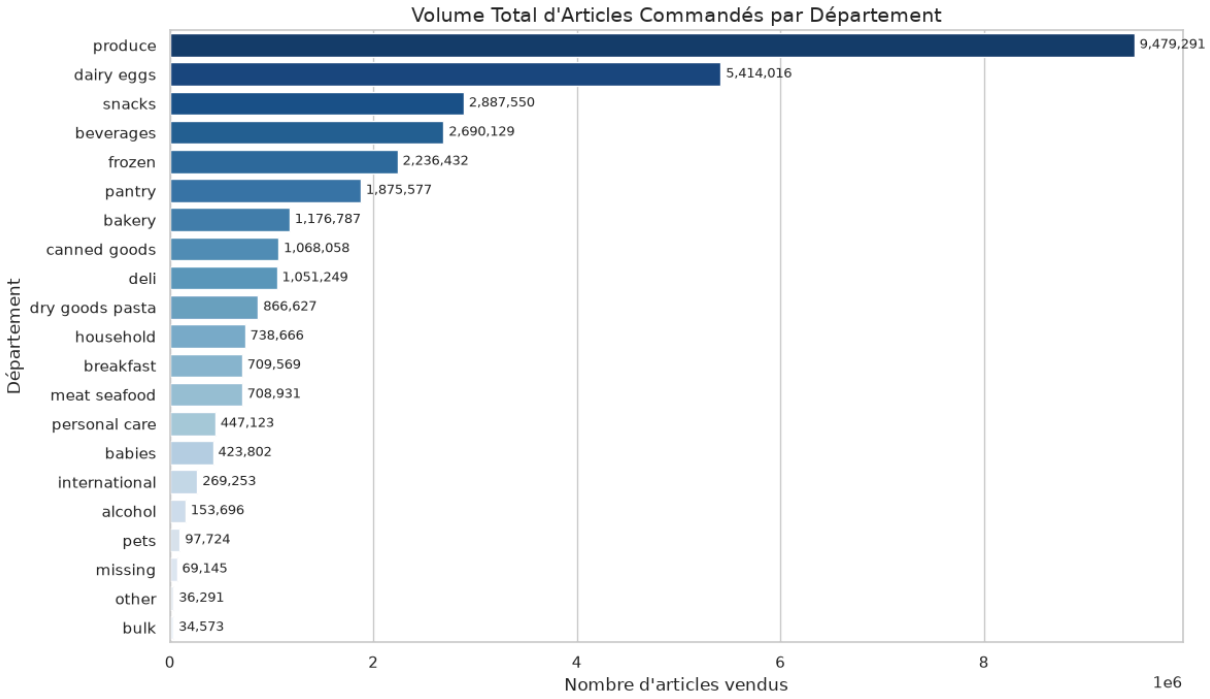




In [7]:
```
# Volume par département
op_deps = order_products.merge(products[['product_id', 'department_id']], or
                        .merge(departments, on='department_id')

dep_counts = op_deps['department'].value_counts().reset_index()
dep_counts.columns = ['department', 'total_items']

plt.figure(figsize=(12, 7))
sns.barplot(data=dep_counts, x='total_items', y='department', hue='departmer
plt.title("Volume Total d'Articles Commandés par Département", fontsize=14)
plt.xlabel("Nombre d'articles vendus")
plt.ylabel("Département")
for i, v in enumerate(dep_counts['total_items']):
    plt.text(v + 50000, i, f"{v:,}", va='center', fontsize=9)
plt.tight_layout()
plt.show()
```



6. Analyse du Réapprovisionnement & Taux de Réachat (*Reorder Rate*)

Le réachat est l'indicateur clé de la fidélité des consommateurs.

- Quel est le taux global de réachat ?
- Quels départements ont le plus fort taux de réachat ?
- La position d'un article dans le panier (`add_to_cart_order`) influence-t-elle sa probabilité d'être un réachat ?

In [8]:

```
overall_reorder_rate = order_products['reordered'].mean() * 100
print(f"📊 Taux de réachat global (Reorder Rate) : {overall_reorder_rate:.2f

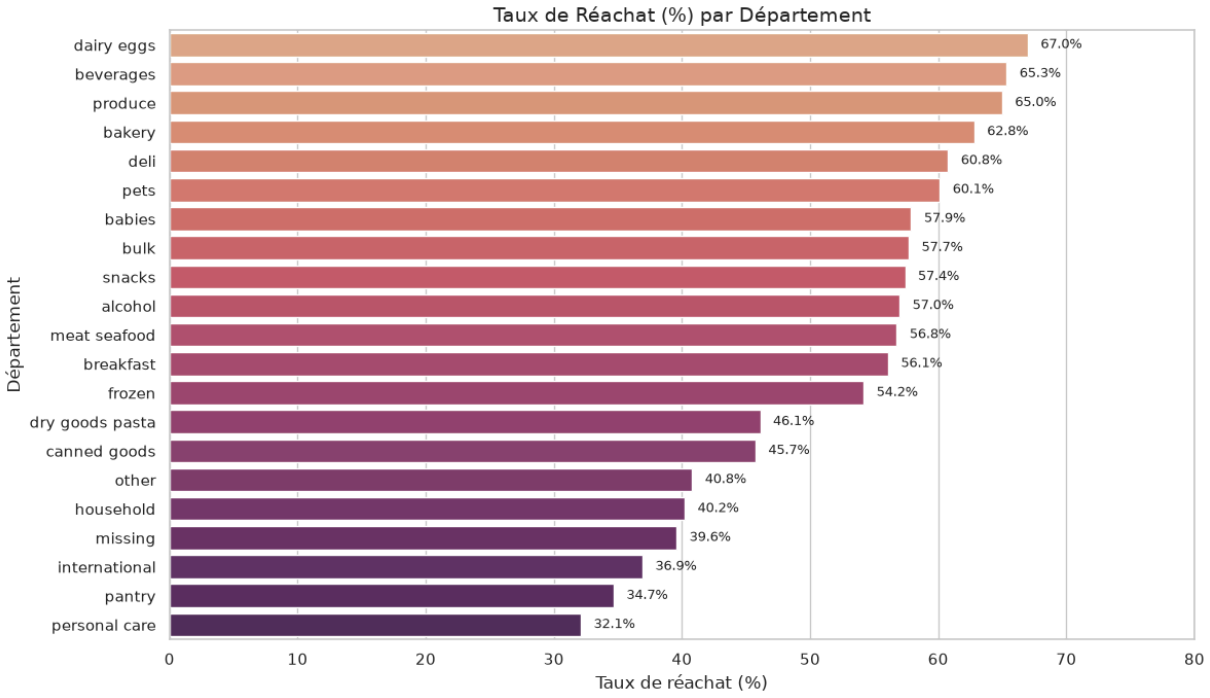
# Taux de réachat par département
dep_reorder = op_deps.groupby('department')['reordered'].agg(['count', 'mean
dep_reorder.columns = ['department', 'total_items', 'reorder_rate']
dep_reorder['reorder_rate_pct'] = dep_reorder['reorder_rate'] * 100
dep_reorder = dep_reorder.sort_values(by='reorder_rate_pct', ascending=False

plt.figure(figsize=(12, 7))
sns.barplot(data=dep_reorder, x='reorder_rate_pct', y='department', hue='dep
plt.title("Taux de Réachat (%) par Département", fontsize=14)
plt.xlabel("Taux de réachat (%)")
plt.ylabel("Département")
plt.xlim(0, 80)

for i, v in enumerate(dep_reorder['reorder_rate_pct']):
    plt.text(v + 1, i, f"{v:.1f}%", va='center', fontsize=9)
```

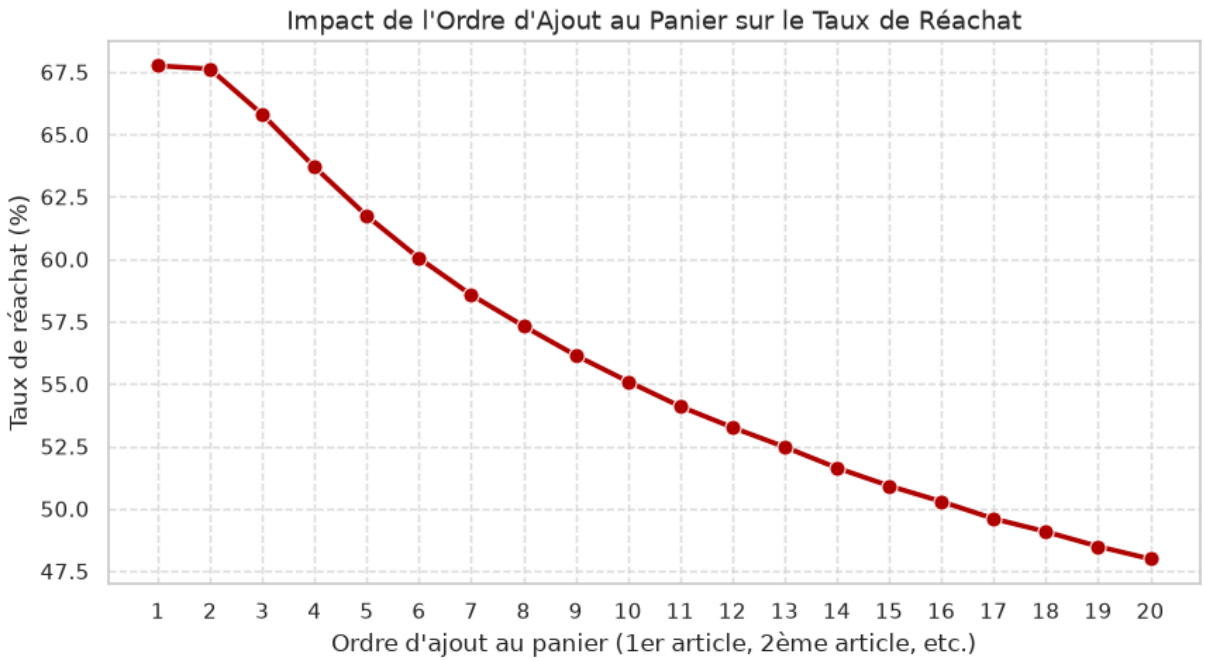
```
plt.tight_layout()
plt.show()
```

📊 Taux de réachat global (Reorder Rate) : 58.97%



In [9]:
```
# Relation position dans le panier vs réachat
cart_reorder = order_products[order_products['add_to_cart_order'] <= 20].grc
cart_reorder = cart_reorder.reset_index()

plt.figure(figsize=(10, 5))
sns.lineplot(data=cart_reorder, x='add_to_cart_order', y='reordered', marker
plt.title("Impact de l'Ordre d'Ajout au Panier sur le Taux de Réachat", font
plt.xlabel("Ordre d'ajout au panier (1er article, 2ème article, etc.)")
plt.ylabel("Taux de réachat (%)")
plt.xticks(range(1, 21))
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()
```


Impact de l'Ordre d'Ajout au Panier sur le Taux de Réachat
Taux de réachat (%)
67.5
65.0
62.5
60.0
57.5
55.0
52.5
50.0
47.5
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
Ordre d'ajout au panier (1er article, 2ème article, etc.)